# 实验：动态分区分配算法模拟

## 一、数据结构及符号说明

程序通过链表管理内存分区。核心结构体定义如下： `MemoryBlock` 表示一个内存分区，成员包括起始地址 `start` 、块大小 `size` 以及空闲标志 `free` （表示该分区是否空闲）。例如：

```
struct MemoryBlock {
    int start;   // 分区起始地址
    int size;    // 分区大小
    bool free;   // 是否空闲
    MemoryBlock(int s, int sz): start(s), size(sz), free(true) {}
};
```

程序使用一个单链表（或数组）保存所有空闲分区，按起始地址递增排序。初始时，整个可用内存视为一个大空闲块。系统中还维护已分配分区的信息（如进程 ID、起始地址、大小）以便释放时查找。空闲分区通过链表（或数组）连接，在分配时从空闲链表中查找满足条件的分区。

## 二、动态分区分配算法原理与实现

- **首次适应算法（First Fit）**：从地址最低的空闲分区开始顺序查找，遇到第一个能满足请求大小的分区时，立即进行分配。若该分区大小大于所需，则将其拆分：保留一个大小为“原大小减去请求大小”的空闲块（保留起始地址不变），并将剩余尾部分配给进程。例如，对一个大小为 `req` 的申请，若找到空闲块 `(start, size)` 且 `size>=req` ，则分配区间为 `[start+size-req, start+size)` ，并将原空闲块缩小为 `(start, size-req)` 。如果分配后空闲块大小变为 0，则将该块从空闲链表中移除。
- **最佳适应算法（Best Fit）**：遍历所有空闲分区，找到能满足请求的所有分区中**大小最小**的那一个进行分配。其核心逻辑是在每次分配前都比较全表，记录最小且足够的块的索引。分配方式同首次适应（分配尾部，必要时拆分空闲块）。由于总是使用最小的合适块，该算法尽可能减少浪费，但寻找过程较为耗时。
- **最坏适应算法（Worst Fit）**：遍历所有空闲分区，选择**最大的**空闲区来分配。即先找出当前最大的空闲块，如果它能满足请求（ `size>=req` ），则在该块末尾分配；否则分配失败。这样做的目的是尽量留下其他较大空闲区，避免产生过小的碎片。实现时只需记录最大块的索引并分配，同样可能拆分空闲块。

**内存回收**：当某进程释放内存时，系统将该分区重新插入空闲链表，并检查与相邻区是否相连以合并。具体地，先在已分配列表中查找对应进程的分区 `(start, size)` ，然后在空闲链表插入新的空闲

节点，并将链表按地址排序；接着遍历空闲链表，若某空闲块的末端恰好与下一个空闲块的起始地址相连，就将它们合并为一个更大的空闲块。这样可以有效缓解外部碎片。

# 三、程序源码

```
#include <iostream>
#include <sstream>
#include <list>
#include <climits>

using namespace std;

struct MemoryBlock
{
    int start;
    int end;
    bool is_free;
    int process_id;

    MemoryBlock(int s, int e, bool free, int pid)
        : start(s), end(e), is_free(free), process_id(pid) {}
};

list<MemoryBlock> memory;

void allocate(list<MemoryBlock>::iterator it, int size, int pid)
{
    int original_start = it->start;
    int original_end = it->end;
    int allocated_end = original_start + size - 1;

    MemoryBlock allocated(original_start, allocated_end, false, pid);
    it = memory.erase(it);
    memory.insert(it, allocated);

    if (allocated_end + 1 <= original_end)
    {
        memory.insert(it, MemoryBlock(allocated_end + 1, original_end, true, -1));
    }
}

void merge_blocks(list<MemoryBlock>::iterator it)
{
    bool merged;
    do
    {
        merged = false;
        if (it != memory.begin())
```

```
        {
            auto prev_it = prev(it);
            if (prev_it->is_free && prev_it->end + 1 == it->start)
            {
                prev_it->end = it->end;
                memory.erase(it);
                it = prev_it;
                merged = true;
            }
        }

        auto next_it = next(it);
        if (next_it != memory.end() && it->is_free && next_it->is_free &&
            it->end + 1 == next_it->start)
        {
            it->end = next_it->end;
            memory.erase(next_it);
            merged = true;
        }
    } while (merged);
}

string get_memory_state()
{
    stringstream ss;
    bool first = true;
    for (const auto &block : memory)
    {
        if (!first)
            ss << "/";
        first = false;
        ss << block.start << "-" << block.end << ".";
        if (block.is_free)
            ss << "0";
        else
            ss << "1." << block.process_id;
    }
    return ss.str();
}

int main()
{
    int algorithm, initial_memory;
    cin >> algorithm >> initial_memory;
    memory.emplace_back(0, initial_memory - 1, true, -1);
```

```
string line;
cin.ignore();
while (getline(cin, line))
{
    if (line.empty())
        continue;

    int seq, pid, op, size;
    sscanf(line.c_str(), "%d/%d/%d/%d", &seq, &pid, &op, &size);

    if (op == 1)
    { // Allocation
        bool success = false;
        list<MemoryBlock>::iterator target_it = memory.end();

        if (algorithm == 1)
        { // First-fit
            for (auto it = memory.begin(); it != memory.end(); ++it)
            {
                if (it->is_free && (it->end - it->start + 1) >= size)
                {
                    target_it = it;
                    success = true;
                    break;
                }
            }
        }
        else if (algorithm == 2)
        { // Best-fit
            int min_size = INT_MAX;
            for (auto it = memory.begin(); it != memory.end(); ++it)
            {
                if (it->is_free)
                {
                    int block_size = it->end - it->start + 1;
                    if (block_size >= size && block_size < min_size)
                    {
                        min_size = block_size;
                        target_it = it;
                        success = true;
                    }
                }
            }
        }
        else
        { // Worst-fit
```

```
                int max_size = -1;
                for (auto it = memory.begin(); it != memory.end(); ++it)
                {
                    if (it->is_free)
                    {
                        int block_size = it->end - it->start + 1;
                        if (block_size >= size && block_size > max_size)
                        {
                            max_size = block_size;
                            target_it = it;
                            success = true;
                        }
                    }
                }
            }

            if (success)
                allocate(target_it, size, pid);
        }
        else if (op == 2)
        { // Deallocation
            for (auto it = memory.begin(); it != memory.end(); ++it)
            {
                if (!it->is_free && it->process_id == pid &&
                    (it->end - it->start + 1) == size)
                {
                    it->is_free = true;
                    merge_blocks(it);
                    break;
                }
            }
        }

        cout << seq << "/" << get_memory_state() << endl;
    }
}
```

# 四、测试方法与结果分析

**输入格式说明**：程序首先输入一个算法名（`first-fit`、`best-fit` 或 `worst-fit`），随后逐行输入操作。每行操作以字符标识：`in` 表示申请内存、`out` 表示释放内存，对应参数分别为进程 ID 和申请大小或仅进程 ID。例如：

```
5              // 操作数
first-fit      // 选择算法
in 1 400       // 进程1申请400K
in 2 100       // 进程2申请100K
in 3 200       // 进程3申请200K
out 1          // 进程1释放
in 4 150       // 进程4申请150K
```

**测试步骤示例**：选择 **首次适应** 算法，假设初始内存 1024K。以如下请求序列为例：

```
first-fit
in 1 400
in 2 100
in 3 200
in 4 10
out 3
out 1
in 5 100
```

程序运行后每步状态可能输出（示例）：

```
// 进程1分配400K
Process 1 allocated [624,1024)
当前空闲区：(0,624)

// 进程2分配100K
Process 2 allocated [524,624)
当前空闲区：(0,524)

// 进程3分配200K
Process 3 allocated [324,524)
当前空闲区：(0,324)

// 进程4分配10K
Process 4 allocated [314,324)
当前空闲区：(0,314)

// 进程3释放
Process 3 freed [324,524)
当前空闲区：(0,314) (324,200)

// 进程1释放
Process 1 freed [624,1024)
当前空闲区：(0,314) (324,200) (624,400)

// 进程5分配100K
Process 5 allocated [214,314)
当前空闲区：(0,214) (324,200) (624,400)
```

输出中每行显示分配或释放的操作，以及当前的空闲块列表。**空闲区**以 (起始, 大小) 形式列出。

**结果分析**：以该示例为例，最终空闲块为大小 214、200、400K，总共 814K，但最大连续空闲块只有 400K。假设接下来进程6申请 300K，由于没有单个块≥300K（分块为214、200、400），则分配失败。这反映出**外部碎片**问题：虽然总空闲814K，但没有足够大的连续空间来满足请求。比较三种算法的效果：

- **首次适应** 通常从低地址开始分配，低地址的小块更易被使用，较大的块保留在高地址；文献指出它在整体效果上往往较优。
- **最佳适应** 倾向使用最小的空闲区，会产生很多较小碎片以供后续大作业使用，但同时可能增加搜索开销。
- **最坏适应** 经常切割最大的空闲区，保留了多个较大区，但可能会留下多个**中等大小碎片**，最终导致无法满足中等大小请求。
  具体测试中，三种算法的内存利用效率可通过“已用内存/总内存”比值来衡量（或外部碎片大小等）。例如上例，首次适应最终使用210K（进程2、4、5共用空间）占1024K的 ~20.5%，碎片化较严重。将相同请求应用于其他算法，可以观察到不同碎片分布：最佳适应可能使用更紧凑的块

（在此例中仍导致相似结果），而最坏适应如果优先用 624K 大块给进程1，可能会保留不一样的碎片。总之，不同算法下空闲块的划分和利用效率不同；当分配失败时，应诊断为**碎片过大**或**内存不足**。

# 五、实验经验与体会

通过本次实验，加深了对操作系统**动态分区分配**的理解。编程过程中学习了如何维护空闲块链表、实现分区的拆分与合并逻辑，以及三种算法的区别。实验体会包括：首次适应实现简单高效，对多数场景表现良好；最佳适应能降低每次的内存浪费但搜索开销大；最坏适应可以保留较大空闲区，但可能造成中等碎片。总体而言，此实验使我深入理解了内存分配与碎片问题，加深了对操作系统内存管理策略的认识，并锻炼了用链表等数据结构模拟操作系统功能的能力。

**参考资料：** 动态分区分配算法的原理和实现可参考相关教材和网络资源等。实验报告中使用的算法逻辑、数据结构设计与上述资料保持一致。